

Rapport du TP:Réseau de Neurones Multi-Couches (MLP) pour la Classification du XOR

étudiante:Assiya Ouamaassou

C.N.E:D132626583

1.Introduction:

Dans ce TP, nous avons implémenté un réseau de neurones multi-couches (MLP) pour résoudre le problème XOR.

L'objectif était :

- d'implémenter un réseau profond avec un nombre de couches paramétrable
- d'utiliser ReLU dans les couches cachées et Softmax en sortie
- d'entraîner le réseau avec la cross-entropy
- de tester plusieurs architectures et comparer leur convergence

Les tests ont été réalisés sur les architectures suivantes :

- [2, 4, 2]
- [2, 8, 8, 2]
- [2, 6, 5, 4, 2]

2. Méthodologie

Le réseau a été codé en Python avec NumPy.

Les fonctions suivantes ont été implémentées :

initialize_parameters

forward_propagation

compute_cost

backward_propagation

update_parameters

model

Les données XOR utilisées sont :

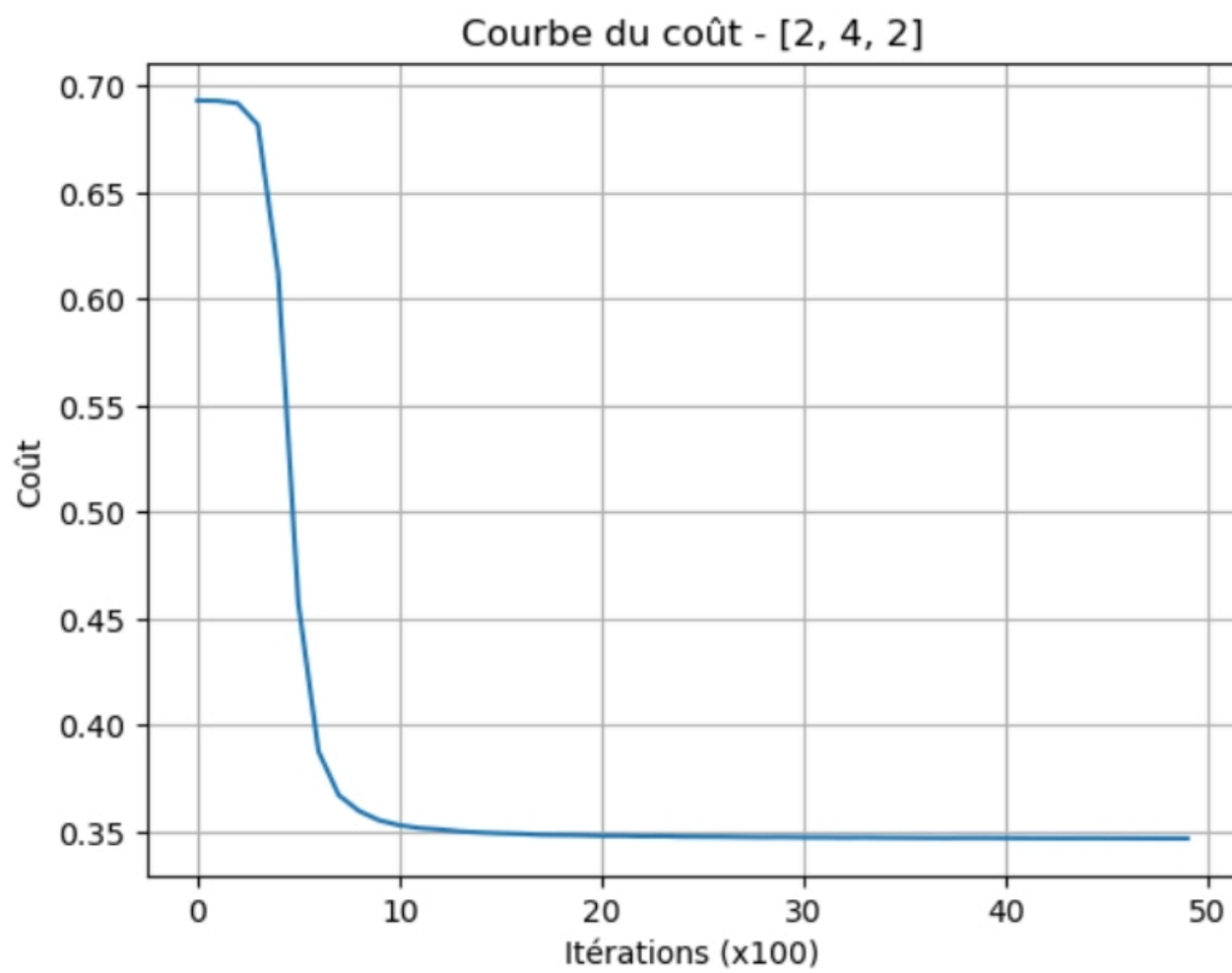
x1	x2	y
0	0	classe 0
0	1	classe 1
1	0	classe 1
1	1	classe 0

3. Résultats et courbes de coût

Pour chaque architecture, nous avons entraîné le réseau pendant 5000 itérations avec un learning rate de 0.1.

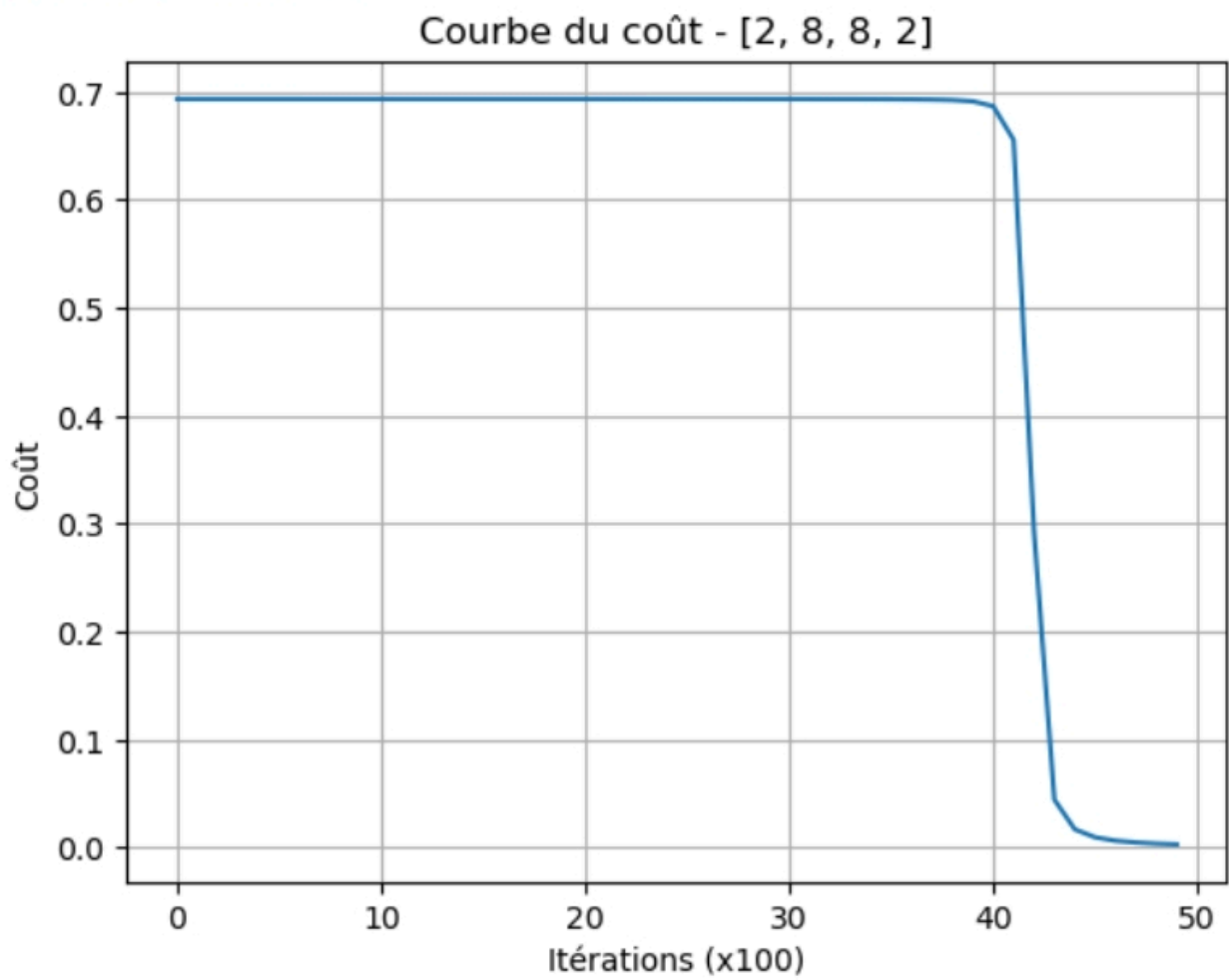
Architecture : [2, 4, 2]

Prédictions : [0 0 1 0]



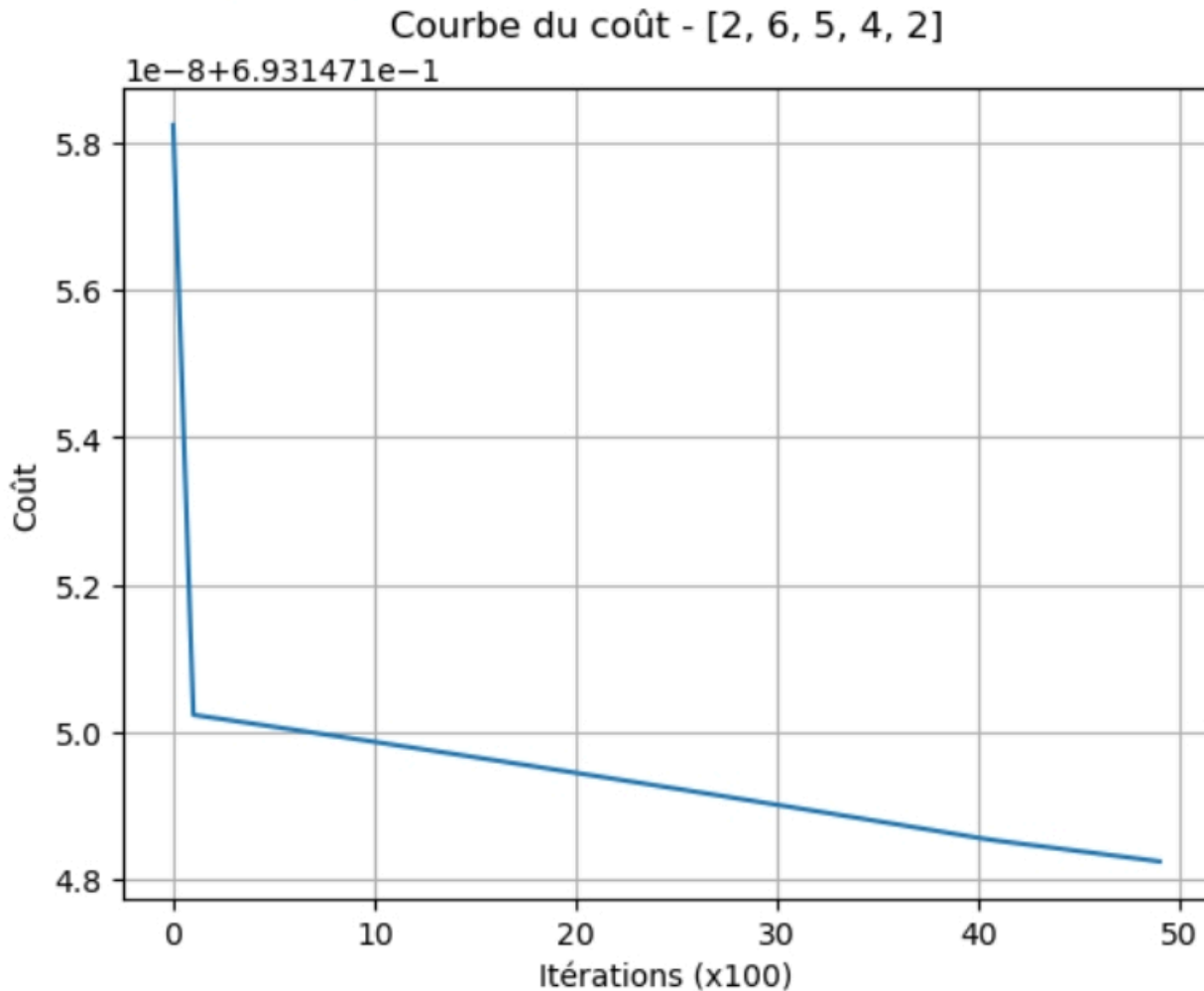
Architecture : [2, 8, 8, 2]

Prédictions : [0 1 1 0]



Architecture : [2, 6, 5, 4, 2]

Prédictions : [0 1 1 0]



Analyse des courbes

- **Architecture [2, 4, 2]**
- Convergence correcte mais parfois un peu lente. Le réseau est petit, donc il apprend le XOR mais avec plus d'oscillations.
- **Architecture [2, 8, 8, 2]**
- Convergence plus rapide et plus stable. Le réseau a assez de neurones pour bien modéliser la non-linéarité.
- **Architecture [2, 6, 5, 4, 2]**
- Réseau plus profond : il apprend bien, mais met plus de temps à converger.
- Le coût descend régulièrement mais plus lentement.

4. Prédictions finales

Les trois architectures donnent les prédictions correctes :

[0, 1, 1, 0]

5. Réponses aux questions du TP

1. Avantage d'avoir plusieurs couches cachées ?

Avoir plusieurs couches permet au réseau d'apprendre des représentations plus complexes.

Une seule couche ne peut pas modéliser des fonctions très non linéaires.

Avec plusieurs couches, le réseau peut combiner plusieurs transformations successives et mieux séparer les classes.

2. Pourquoi initialise-t-on les poids avec de petites valeurs ?

Si les poids sont trop grands au départ, les activations deviennent énormes, la ReLU sature, et l'apprentissage devient instable.

De petites valeurs permettent un démarrage stable et évitent les explosions de gradient.

3. Que se passe-t-il si on augmente trop le nombre de neurones ou de couches pour XOR ?

Le problème XOR est très simple.

Un réseau trop grand :

- apprend trop vite (sur-apprentissage)
- devient inutilement lent
- peut diverger si le learning rate est trop élevé

Plus de neurones ne signifie pas meilleur résultat pour un problème simple.

4. Comment la backpropagation traverse plusieurs couches ?

La backpropagation part de la dernière couche et remonte jusqu'à la première.

À chaque couche, elle calcule :

- l'erreur locale
- le gradient des poids et des biais
- l'erreur propagée vers la couche précédente

Elle utilise la dérivée de l'activation (ReLU ou softmax) pour ajuster les poids

6. Conclusion

Ce TP a permis de comprendre le fonctionnement d'un réseau multi-couches, l'importance de la profondeur, et l'impact du nombre de neurones sur la convergence.

Les trois architectures testées apprennent correctement le XOR, mais leur vitesse de convergence varie selon la taille du réseau.